INTEGRATION.md

> How each teammate plugs in their own Spotify producer (with *different* JSON fields), wires Kafka → Mongo, adds a Streamlit tab, and runs everything **Docker-only**.

---

0) What you need installed

- **Docker** + **Docker Compose** (already covered by the `scripts/install-docker-ubuntu.sh` for Ubuntu).
- **No local Python/venv required** — everything runs in containers.

---

1) Naming conventions (important and consistent)

We use a **per-teammate prefix** called `APP_PREFIX`. This controls *everything*:

- **Kafka topics**
  - `${APP_PREFIX}_recent_events`
  - `${APP_PREFIX}_artist_market_top_tracks` (or your custom topics)
- **Mongo collections**
  - `${APP_PREFIX}_recent_events`
  - `${APP_PREFIX}_artist_market_top_tracks`
  - Add more as needed — keep the same prefix.
- **Env files** live in `src/envs` as `.env.<prefix>`, e.g. `src/envs/.env.alex`.
- **Producer module** in `src/producers/`: `<prefix>_producer.py`.
- **Streamlit tab** in `src/app/tabs/`: `tab_<prefix>.py`.
- **Makefile targets**: `<prefix>-env` and `<prefix>-demo`.

> TL;DR: pick a short lowercase prefix (e.g. `alex`) and use it everywhere.

---

2) Create your personal env file

Create **`src/envs/.env.<prefix>`** and fill this (adjust values):

```
# ========================================
# <PREFIX> — Docker-only environment configuration
# ========================================

# Who am I?
APP_PREFIX=<prefix>          # e.g. alex

# Spotify credentials (yours)
CLIENT_ID=...
CLIENT_SECRET=...
USERNAME=...
REDIRECT_URI=http://127.0.0.1:8888/callback
```

```
# Optional: pick a market for Top 10 lookups (ISO code)
MARKET_OVERRIDE=US

# Spotipy compatibility
SPOTIPY_CLIENT_ID=${CLIENT_ID}
SPOTIPY_CLIENT_SECRET=${CLIENT_SECRET}
SPOTIPY_REDIRECT_URI=${REDIRECT_URI}

# Debug / feature flags
DEBUG=true
KAFKA_ENABLED=true

# Kafka inside Docker network
KAFKA_BOOTSTRAP=kafka:19092

# Topics (use your prefix!)
KAFKA_TOPICS=${APP_PREFIX}_recent_events,${APP_PREFIX}_artist_market_top_tr
acks
KAFKA_TOPIC_ROUTES=${APP_PREFIX}_recent_events:events,${APP_PREFIX}_artist_
market_top_tracks:top10

# Mongo inside Docker network
MONGO_URL=mongodb://root:example@mongo:27017/?authSource=admin
MONGO_DB=spotify_db

# Collections (use your prefix!)
MONGO_COLL_EVENTS=${APP_PREFIX}_recent_events
MONGO_COLL_TOP10=${APP_PREFIX}_artist_market_top_tracks
GENERIC_COLL_NAMESPACE=${APP_PREFIX}__topic__

# Which producer module to run (see §3)
PRODUCER_ENTRY=<prefix>_producer

# Spark (optional; leave as-is if unsure)
SPARK_KAFKA_TOPICS=
SPARK_TRIGGER_SECS=10
SPARK_WATERMARK_MIN=5
SPARK_COLL_EVENTS=
SPARK_COLL_RAW=
SPARK_MODE=top_artists
SPARK_TOPN=10
SPARK_FEATURE=track_duration_ms
SPARK_GROUP=market_used
SPARK_PACKAGES=org.apache.spark:spark-sql-kafka-0-
10_2.12:3.5.1,org.apache.kafka:kafka-clients:3.5.2

# Streamlit port (host)
PORT=8501
```

> **Never commit secrets** — `.env` files are git-ignored by default.

---

3) Add your producer

Create `src/producers/<prefix>_producer.py`.

Minimal skeleton (send whatever JSON you fetch — your schema can differ!):

```
# src/producers/<prefix>_producer.py
import json, os
from dotenv import load_dotenv
from kafka import KafkaProducer
import spotipy
from spotipy.oauth2 import SpotifyOAuth

load_dotenv()

BOOTSTRAP = os.getenv("KAFKA_BOOTSTRAP", "kafka:19092")
TOPIC_EVENTS = os.getenv("MONGO_COLL_EVENTS") or f"
{os.getenv('APP_PREFIX')}_recent_events"
TOPIC_TOP10  = os.getenv("MONGO_COLL_TOP10")  or f"
{os.getenv('APP_PREFIX')}_artist_market_top_tracks"
DEBUG = os.getenv("DEBUG", "false").lower() == "true"

def _producer():
    return KafkaProducer(
        bootstrap_servers=BOOTSTRAP,
        value_serializer=lambda v: json.dumps(v).encode("utf-8"),
        key_serializer=lambda v: json.dumps(v).encode("utf-8") if v is not
None else None,
        linger_ms=50
    )

def auth_spotify():
    return spotipy.Spotify(auth_manager=SpotifyOAuth(
        client_id=os.getenv("CLIENT_ID"),
        client_secret=os.getenv("CLIENT_SECRET"),
        redirect_uri=os.getenv("REDIRECT_URI"),
        scope="user-read-recently-played user-read-playback-state"
    ))

def run():
    p = _producer()
    sp = auth_spotify()

    # Example: send “recent plays” (your schema is OK as-is)
    recent = sp.current_user_recently_played(limit=50)
    for item in recent.get("items", []):
        event = {
            "prefix": os.getenv("APP_PREFIX"),
            "type": "recent_play",
            "played_at": item["played_at"],
            "track_name": item["track"]["name"],
            "artist_names": [a["name"] for a in item["track"]["artists"]],
            "raw": item,  # keep raw payload (optional)
```

```
            }
            p.send(TOPIC_EVENTS, value=event)
        if DEBUG: print(f"[KAFKA] sent recent events -> {TOPIC_EVENTS}")

        # Example: your custom payload (e.g., danceability/energy/etc.) to
    TOPIC_TOP10 or a new topic
        # payload = {...}
        # p.send(TOPIC_TOP10, value=payload)

        p.flush()

    if __name__ == "__main__":
        run()
```

Important:

- **Your JSON schema can be different** (different fields/structure).
- The consumer and your Streamlit tab should read **your** schema from **your** collections.
- Keep messages **valid JSON** and **newline-free**.

---

4) Kafka topics & Mongo collections

No code changes needed if you follow the env names:

- Topics are created with:

  ```
  make kafka-init-from-env
  ```

- The consumer writes to `${APP_PREFIX}_*` collections by default (see §5 for mapping).

---

5) Consumer mapping (topic → collection)

Generic consumer: **`src/consumers/kafka_consumer.py`**

It:

- reads JSON from Kafka,
- picks Mongo collection name based on topic,
- inserts the full document.

To add a **custom topic → collection** mapping, extend your env:

```
KAFKA_TOPICS=${APP_PREFIX}_recent_events,${APP_PREFIX}_artist_market_top_tr
acks,${APP_PREFIX}_my_feature
KAFKA_TOPIC_ROUTES=${APP_PREFIX}_recent_events:events,${APP_PREFIX}_artist_
market_top_tracks:top10,${APP_PREFIX}_my_feature:my_feature
```

Consumer will write to collection `${APP_PREFIX}_my_feature` automatically.

---

6) Add your Streamlit tab

Create `src/app/tabs/tab_<prefix>.py`:

```
import os, streamlit as st
from pymongo import MongoClient

def get_coll(name: str):
    client = MongoClient(os.getenv("MONGO_URL"))
    db = client[os.getenv("MONGO_DB", "spotify_db")]
    return db[name]

def render():
    st.header(f"{os.getenv('APP_PREFIX').upper()} — My Feature Dashboard")

    coll_events = get_coll(os.getenv("MONGO_COLL_EVENTS") or f"
{os.getenv('APP_PREFIX')}_recent_events")
    coll_top10  = get_coll(os.getenv("MONGO_COLL_TOP10")  or f"
{os.getenv('APP_PREFIX')}_artist_market_top_tracks")

    recent = list(coll_events.find().sort("played_at", -1).limit(50))
    st.subheader("Recent events (last 50)")
    st.code(recent[:3])

    # Example custom collection
    # my_coll = get_coll(f\"{os.getenv('APP_PREFIX')}_my_feature\")
    # rows = list(my_coll.find().limit(20))
    # st.write(rows)
```

The app auto-discovers files in `app/tabs` named `tab_*.py` that expose `render()`.

---

7) Spark (optional)

Spark is already configured. If your schema differs:

1. **Raw JSON mode (easy):** leave as-is; Spark treats Kafka value as string and stores raw docs.
2. **Typed mode:** update `src/spark/streaming_job.py` to parse with your own schema:

```
from pyspark.sql.functions import from_json, col
from pyspark.sql.types import StructType, StructField, StringType, ...

schema = StructType([...])
df_parsed = df.select(from_json(col("value").cast("string"),
schema).alias("j")).select("j.*")
```

Tune with `.env` (`SPARK_MODE`, `SPARK_FEATURE`, etc.) if you use built-in aggregations.

---

8) Makefile targets for your prefix

Open `src/makefile` and add (copy/adapt one of the existing ones):

```
alex-env:
    @cp envs/.env.alex .env
    @echo "[ENV] Loaded .env.alex for Alex"

alex-demo: alex-env
    docker compose up -d zookeeper kafka mongo
    make kafka-init-from-env
    docker compose run --rm -e PRODUCER_ENTRY=alex_producer -e
PYTHONPATH=/app/src producer
    docker compose up -d spark-app app
    @echo "[INFO] Demo up and running for Alex!"
```

Now run `make alex-demo` to go end-to-end.

---

9) Run it (manual steps if you prefer)

From `src/`:

```
# 1) choose your env
make <prefix>-env           # e.g. make alex-env

# 2) bring up infra
docker compose up -d zookeeper kafka mongo

# 3) create topics
make kafka-init-from-env

# 4) run your producer (sends data)
docker compose run --rm -e PRODUCER_ENTRY=<prefix>_producer -e
PYTHONPATH=/app/src producer

# 5) optional: start consumer
docker compose up -d consumer

# 6) start Spark + Streamlit app
docker compose up -d spark-app app
```

Open:

- **Streamlit:** http://localhost:8501
- **Mongo Express:** http://localhost:8081 (user `admin`, pass `secret`)

> If a port is taken, edit host mappings in `src/docker-compose.yml`.

---

10) Common problems / fixes

- **“Cannot connect to the Docker daemon”**

```
sudo systemctl start docker
sudo usermod -aG docker $USER
newgrp docker
```

- **Port already in use (2181 / 9092 / 27017 / 8081 / 8501)**

```
sudo ss -ltnp '( sport = :8081 )'
sudo kill -9 <pid>
```

  Or change host port mapping in `src/docker-compose.yml`.

- **Kafka topics not created**
  Run `make kafka-init-from-env` after Kafka is healthy.

- **Different JSON fields**
  Totally fine. Consumer is schema-agnostic. Your Streamlit tab should read your fields from your collections.

---

11) Security & repo hygiene

- Do **not** commit `.env` files or credentials (already in `.gitignore`).
- Keep each teammate’s secrets in `src/envs/.env.<prefix>`.

---

12) Docker installer script (Ubuntu users)

Run once:

```
src/tools/install-docker-ubuntu.sh
```

It installs Docker CE + Compose and adds your user to the `docker` group.

FAQ: “My JSON fields are different — will it break?”

No. The consumer and Streamlit pattern are **schema-agnostic**.
Send **valid JSON**, use your prefix for topics/collections, and read those docs in your tab. Spark can stay in raw mode until you decide on a typed schema.